

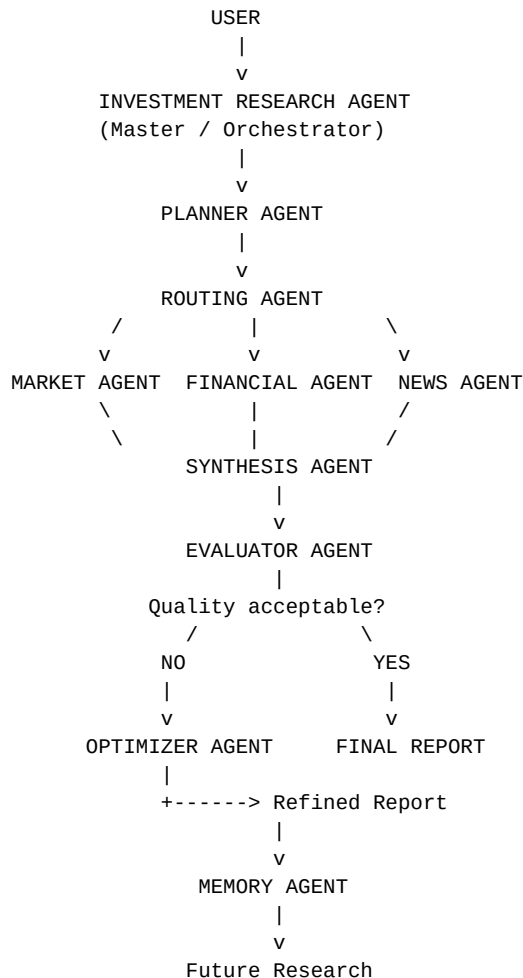
AAI-520 Final Team Project

Multi-Agent Financial Analysis System — Full Architecture

Primary Data Source: Yahoo Finance via yfinance

This architecture is designed to satisfy the project requirements for Agent Functions (planning, dynamic tool use, self-reflection, and learning across runs) and the three required Workflow Patterns (Prompt Chaining, Routing, and Evaluator–Optimizer).

1. Overall System Architecture



2. User Input Layer

The user provides a stock-research request, for example: **Analyze NVDA**. The system identifies the stock symbol, company, and research objective before creating a plan.

3. Investment Research Agent

The Investment Research Agent acts as the master orchestrator. It understands the request, invokes the Planner, coordinates specialist agents, triggers evaluation and optimization, and stores the final findings in memory. It should coordinate the work rather than perform every analysis itself.

4. Planner Agent

The Planner demonstrates the first required Agent Function: **planning research steps for a given stock symbol**.

1. Retrieve current market information.
2. Retrieve historical price data.
3. Retrieve financial statements and company information.
4. Calculate important financial metrics.
5. Retrieve recent financial news.
6. Classify and analyze news.
7. Extract important events and findings.
8. Combine market, financial, and news findings.
9. Generate a research report.
10. Evaluate and refine the report.
11. Store findings in memory.

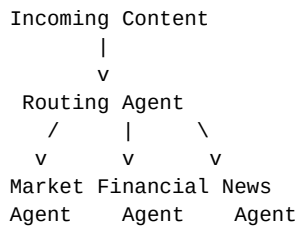
5. Yahoo Finance Tool Layer

Yahoo Finance through `yfinance` is the primary data source. The system can expose focused tools so agents dynamically request only the information needed for a task.

Tool Category	Example Tools	Purpose
Market	<code>get_current_price()</code>	Current market price
Market	<code>get_historical_prices()</code>	Historical price/volume data
Market	<code>calculate_returns()</code>	Return calculations
Market	<code>calculate_volatility()</code>	Risk/volatility metrics
Financial	<code>get_income_statement()</code>	Revenue, profit, EPS
Financial	<code>get_balance_sheet()</code>	Assets, liabilities, cash, debt
Financial	<code>get_cash_flow()</code>	Cash-flow analysis
Company	<code>get_company_info()</code>	Company profile and metadata
News	<code>get_latest_news()</code>	Recent financial news

6. Routing Agent

The Routing Agent determines which specialist should process each task or piece of content. This directly implements the required **Routing Workflow Pattern**.



Examples: price movement → Market Agent; quarterly revenue/EPS → Financial Agent; AI partnership announcement → News Agent.

7. Market Agent

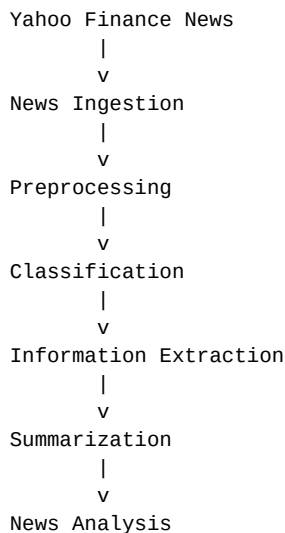
Analyzes market information such as current price, historical prices, volume, returns, 52-week range, moving averages, and volatility. Its output is a structured market-analysis summary.

8. Financial Agent

Analyzes fundamentals using income statements, balance sheets, cash-flow data, company information, and derived metrics such as revenue growth, profit margin, EPS, debt, cash, and free cash flow.

9. News Agent — Prompt Chaining

The News Agent implements the required Prompt Chaining workflow:



Possible classifications include Earnings, Product, Partnership, Management, Regulation, AI/Technology, Market, and Other.

10. Synthesis Agent

The Synthesis Agent combines outputs from the Market, Financial, and News Agents into one coherent research report. A suggested structure is: Company Overview, Market Performance, Financial Performance, Recent News, Key Events, Opportunities, Risks, and Research Summary.

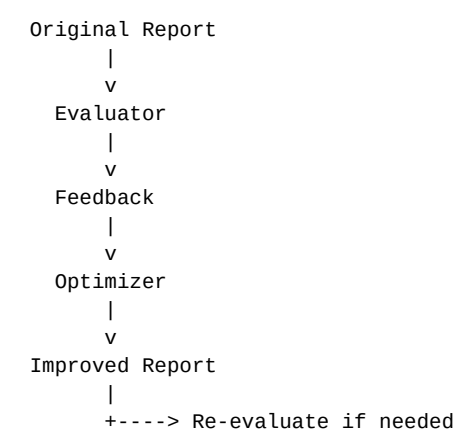
11. Evaluator Agent — Self-Reflection

The Evaluator Agent reviews the generated report against predefined criteria such as completeness, financial analysis, market analysis, news coverage, evidence, consistency, and clarity. It produces structured feedback that can be used by the Optimizer.

Criterion	Example Score
Completeness	8/10
Financial Analysis	9/10
Market Analysis	7/10
News Analysis	6/10
Evidence	8/10
Consistency	8/10

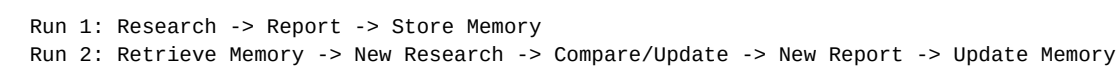
12. Optimizer Agent — Evaluator–Optimizer Workflow

The Optimizer receives the original report and evaluator feedback, then generates an improved version. If the quality score is below a defined threshold, the report is refined and evaluated again. If the report meets the threshold, it proceeds to the final output.

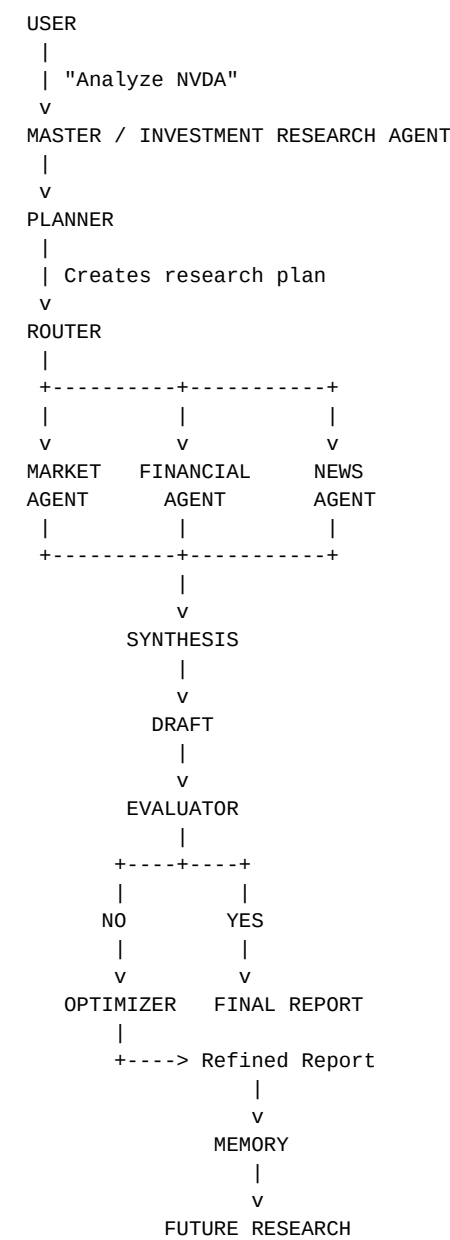


13. Memory Agent — Learning Across Runs

A lightweight JSON or SQLite memory layer can store previous findings, important events, identified risks, evaluator feedback, and report summaries. On later runs, the system retrieves relevant memory and uses it to focus the new research.



14. Complete End-to-End Workflow



15. Mapping to the AAI-520 Rubric

Project Requirement	Architecture Component
Plans research	Planner Agent
Uses tools dynamically	Yahoo Finance Tool Layer
Self-reflects	Evaluator Agent
Learns across runs	Memory Agent
Prompt Chaining	News Agent Pipeline
Routing	Routing Agent
Evaluator–Optimizer	Evaluator + Optimizer
Agent Functions	Planner + Tools + Evaluator + Memory
Workflow Patterns	Chaining + Routing + Evaluator/Optimizer
Code	Python + Jupyter Notebook + GitHub

16. Suggested GitHub Project Structure

```
multi-agent-financial-analysis/  
|  
+-- README.md  
+-- notebooks/  
|   +-- final_project.ipynb  
|  
+-- agents/  
|   +-- planner_agent.py  
|   +-- market_agent.py  
|   +-- financial_agent.py  
|   +-- news_agent.py  
|   +-- routing_agent.py  
|   +-- synthesis_agent.py  
|   +-- evaluator_agent.py  
|   +-- optimizer_agent.py  
|   +-- memory_agent.py  
|  
+-- tools/  
|   +-- yahoo_finance.py  
|   +-- news_tools.py  
|   +-- financial_metrics.py  
|  
+-- workflows/  
|   +-- prompt_chaining.py  
|   +-- routing.py  
|   +-- evaluator_optimizer.py  
|  
+-- memory/  
|   +-- research_memory.json  
|  
+-- data/  
+-- requirements.txt  
+-- .gitignore
```

17. Recommended Two-Member Contribution Structure

Area	Member 1	Member 2
Agent/Workflow Development	Planner, Router, Market/Financial Agents	News, Synthesis, Evaluator/Optimizer, Memory
Integration	Workflow integration and tool testing	End-to-end testing and evaluation
Documentation	Architecture and agent documentation	Evaluation, results, and notebook documentation
Final Deliverable	Notebook/code review	Notebook/code review

Implementation note: The system should emphasize research, evidence, and agent coordination rather than making the project primarily a stock-price prediction system. Prediction can be an optional extension, but the required agent functions and workflow patterns should remain the core of the project.